Soup Kernel Challenge: в 5 раз меньше памяти и на 23% быстрее

Собственные Triton-ядра для дообучения LLM на бесплатной Kaggle T4

Команда claudemaxing · Decentrathon 6.0

Проблема: бесплатная T4 упирается в память и время

~80 с

дообучение Qwen2.5-1.5B (NF4, LoRA) через Soup

14 345

МиБ пик памяти из ~15 ГБ

- гигабайтные логиты по словарю из 152 тысяч токенов
- fp32-копии и лишние приведения типов, мелкие поэлементные операции
- распаковка 4-битных весов и вычисления на pad-токенах

Что мы сделали: собственные Triton-ядра

Операция	Что изменили
Логиты и потеря	слитое ядро по словарю кусками, полные логиты не создаются
SwiGLU	слияние и пересчёт в backward вместо хранения
RMSNorm	слитый forward/backward, одна fp16-копия на всех
RoPE	одно ядро вместо нескольких операций
Распаковка NF4	своё ядро, побитно как bitsandbytes
Dropout в LoRA	маска упакована в биты
Эмбеддинг, attention	fp16 без потерь; SDPA без копий k/v

Главная идея: без pad-токенов

Было: батч дополняется до самой длинной строки



Стало: считаем только реальные токены, одной строкой



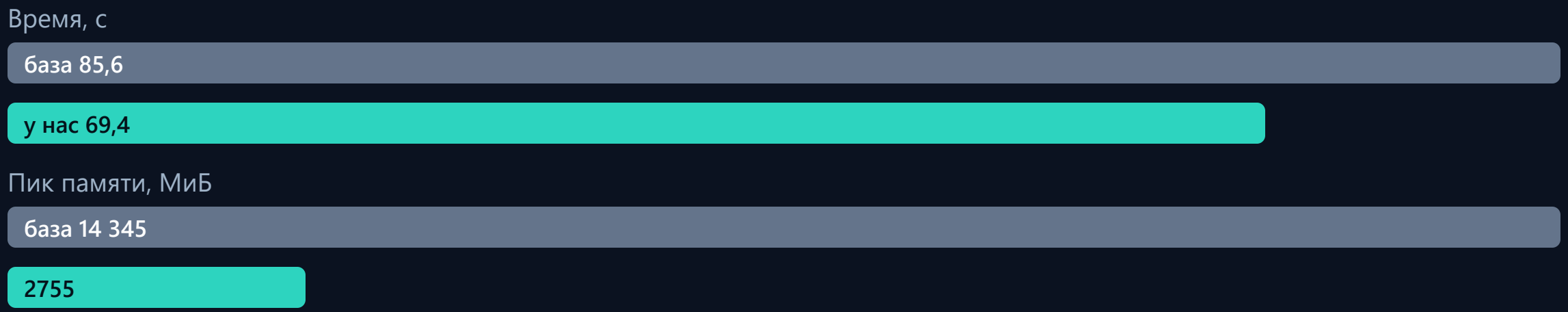
- маска даёт каждому токену то же внимание, что и в базе: только своя строка, только до своей позиции
- строки и батчи не меняются, разрешено условиями
- меньше вычислений и меньше активаций
- 77,9 с → **69,4 с**, память 2839 → **2755** МиБ

Результаты на официальной проверке

1,233×
быстрее
85,6 с → 69,4 с

5,207×
меньше памяти
14 345 → 2755 МиБ

−0,008%
eval loss к базе
допуск: не хуже +0,5%



Kaggle T4, база и решение измерены в одной сессии · все 10 проверок check_rules.py пройдены · наши ядра 15,3% времени

Качество и честность

- задача не менялась: модель, данные, порядок батчей, шаги, LoRA, оптимизатор
- метрики Soup (энтропия, точность, потери по строкам) совпадают с базой
- 79 из 79 тестов Soup проходят с нашими патчами; ядра проверены тестами против torch
- готовые ядерные библиотеки не используются, источники идей указаны
- **Не вышло:** свой flash-attention на Triton оказался в 15 раз медленнее torch SDPA на T4 (нет тензорных ядер для `tl.dot` на sm75), в решение не включён

Ограничения и следующие шаги

Что ограничивает

- фиксированный старт ~24 с и JIT Triton ~6 с
- память: веса NF4 (~645 МиБ), эмбеддинг (445 МиБ), контекст CUDA

Дальше

- attention по строкам для упакованных токенов
- int8-эмбеддинг
- меньше вариантов компиляции Triton
- цель: ~50 с и ~1,8 ГБ